Speaker Notes — Explore Functional Programming in Go

เวิร์กชอป 2 ชม. · 16 สไลด์ · ตัวเลขใน [วงเล็บ] = นาทีบน timeline (0:00–2:00) · เด็ค: ~/src/go-fp/slides/fp-in-go-workshop.pptx

1 Explore Functional Programming in Go

[0:00] เปิดด้วยคำถาม: เคยแก้ bug ที่โค้ด 'รันครั้งที่สองผลไม่เหมือนครั้งแรก' ไหม? — นั่นคือ action ที่หลุดมาปน calculation ทั้งงานไม่พูด monad/functor เลย (ใครถามเรื่อง Haskell/monad เลื่อนไปตอบท้ายชั่วโมง)

2 จบ 2 ชั่วโมงนี้ คุณจะได้

[0:02] ย้ำสัญญา: ออกจากห้องไปด้วยทักษะ ไม่ใช่ความรู้ลอย ๆ — เล่า flow วันนี้สั้น ๆ: ดูโค้ด เขียนโค้ด แล้วมีการบ้านไปต่อที่บ้าน (เวลาเริ่มเบี้ยว: middleware เป็นตัวตัดก่อน)

3 แกนคิดของวันนี้: ทุกบรรทัดโค้ดคือ 1 ใน 3 อย่างนี้

[0:05] ให้ผู้เรียนยกตัวอย่าง action/calculation จากงานตัวเองคนละ 1 อย่าง (30 วินาที) — ประเด็น: calculation มักแฝง 'ในหัว' เช่น กติกาส่วนลดที่อยู่ใน if ของ handler ใหญ่ ๆ จนไม่มีใครเห็นว่ามันคือ calculation แยกอยู่

4 ลองอ่านโค้ดนี้

[0:08] อย่าเฉลยเร็ว — จับคู่ชี้บรรทัด 3 นาที แล้วเฉลยบนกระดาน จุดที่ติง: order.Total ถูก mutate (action บน data), กติกา 1000→90% คือ calculation ที่ฝังอยู่, time.Now/print/save คือ action ชัด ๆ คำถามขวาคือสะพานไปหน้าถัดไป

5 โค้ดที่ปนกันทำให้ชีวิตยาก

[0:13] อย่าเพิ่งโชว์วิธีแก้ — ให้เห็นปัญหาก่อน ถามว่าใครเคยเจอ bug 'ผ่านข้อมูลเข้าไปแล้วมันเปลี่ยน' (mutate input) แล้วสะพานไปหน้า refactor

6 จาก Action สู่ Calculation

[0:15] พิมพ์สดหรือสลับหน้าจอ go-fp/02 จุดสำคัญคือ 'ย้ายไปขอบ' — FP ไม่ได้แปลว่ากำจัด side effect แต่จัดที่อยู่ให้มัน ถามต่อ: LogDiscount ยังเทสต์ยากไหม? → ยาก แต่เล็กและไม่มีความรู้ธุรกิจอยู่ข้างในแล้ว

7 ผลลัพธ์: ทดสอบได้โดยไม่ต้อง mock อะไรเลย

[0:25] รันเทสต์ให้เห็นเขียว ตาราง 3 แถวนี้คือ spec ของกติกาส่วนลด — QA/PM อ่านออกด้วย นี่คือเหตุผลที่ calculation คู่ควรกับการถูกแยกออกมา

8 เครื่องมือ 1: ฟังก์ชันเป็นค่า (Higher-order Functions + Closures)

[0:40] ให้ผู้เรียนเดาลายเซ็น Curry2 ก่อนเฉลย (โค้ดเต็ม + เทสต์: go-fp/04) ใครถาม 'ทำไม Go ไม่มี lambda สั้น ๆ' → ตอบตรง ๆ ว่า verbose by design และ (value, error) ทำ chain ยาก

9 กับดักของ closure: loop variable

[0:50] ถ้ามีเวลา รันสดเทียบ Go เก่า/ใหม่ (gotip) เชื่อมกลับกรอบ ACD: goroutine คือ action, ตัวแปรที่ถูกจับร่วมคือ data ที่แชร์โดยไม่ตั้งใจ

10 เครื่องมือ 2: Generics — เขียน Map / Filter / Reduce เองได้

[0:52] แนะให้เปิด fp_test.go เป็นตัวอย่าง TDD — เขียนเทสต์ก่อนแล้วเห็นลายเซ็น generic ทีละขั้น ใครถาม performance: for loop ตรง ๆ ชนะเสมอ ไม่มี magic

11 Functional Options — pattern ที่คุณใช้อยู่แล้ว

[1:05] aha moment ของวัน: คนที่ใช้ zap.New(...WithFields) มาตลอดจะเห็นว่ามันคือ pattern เดียวกับ Adder/Curry2 เพิ่งเรียน — เดินหน้านี้สั้น ๆ แล้วไป middleware

12 Middleware = Function Composition

[1:10] ถ้าเวลาเหลือน้อย ตัดหน้านี้ทิ้งหน้าได้ (พูดประโยคเดียว: 'middleware คือการเอาฟังก์ชันมาคู่กัน ลำดับมีความหมาย') แล้วกระโดดไปของหวาน — ไม่กระทบเรื่องเล่า

13 ของหวาน: Lazy Sequence ด้วย iter.Seq (Go 1.23+)

[1:25] โชว์อย่างเดียว ไม่ลงลึก: yield คืน false = ผู้อ่านสั่งหยุด ทำให้ infinite sequence ปลอดภัย แถมขำ: int ยัง overflow ถ้าดันอ่านไกล — ของจริงไม่มีอะไร infinite หน้านี้ตัดได้ถ้าเวลาโดนตัด แต่เป็นของที่คนจำได้นาน พยายามรักษาไว้

14 สรุป: 3 คำถามกับโค้ดทุกบรรทัด

[1:40] ปิดเนื้อหาด้วยกรอบเดียว: 3 คำถามนี้คือสิ่งที่อยากให้จำกลับบ้าน — การบ้านคือโอกาสฝึกใช้มันหน้าเดียว

15 การบ้าน: แยกคูปองออกจากการส่งอีเมล

[1:48] มอบการบ้าน: ใช้เวลา ~20 นาที ใบ้จุดพลาดบ่อยไว้ด้วย — (1) ลืมรักษาลำดับคูปองเดิม (2) return slice ใหม่แต่ mutate element ข้างใน (ยังไม่ pure) (3) แอบเอาการหา coupon กลับเข้าไปใน action ครั้งหน้าเปิดเฉลยพร้อมกัน

16 อ่านต่อ

เผื่อ Q&A: (1) ทำไม Go ไม่มี pipe operator → verbose by design + (value, error) ทำ chain ยาก (2) copy-on-write แพงไหม → benchmark ใน go-fp/08: struct เล็กต่างกัน ~0.05 ns/op (3) goroutine/channel เข้ากรอบไหน → channel คือ data, การส่ง/รับคือ action (4) ใครถาม monad → วันนี้ไม่จำเป็น แนะนำหนังสือต่อ